

TUGAS PENGENALAN POLA



Disusun Oleh :

Nama : Ghazi Alvin Karim

Nim : 121450123

Mata Kuliah : Pengenalan Pola (RA)

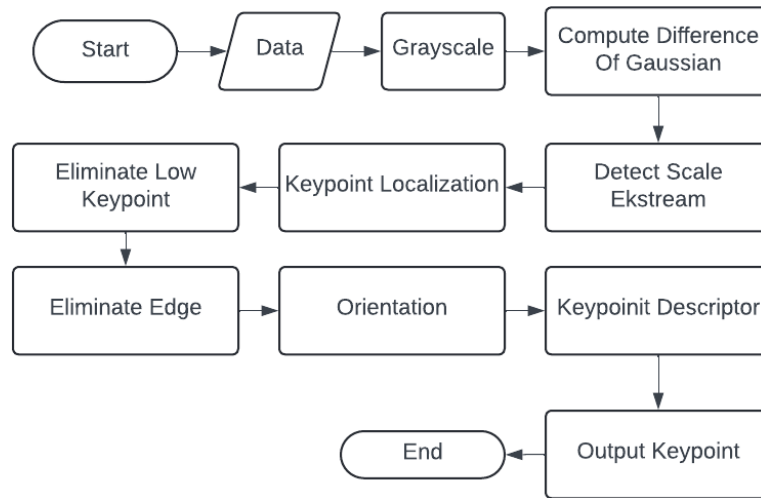
**PROGRAM STUDI SAINS DATA
JURUSAN SAINS
INSTITUT TEKNOLOGI SUMATERA
2024/2025**

- Implementasi deteksi dan pencocokan fitur dengan algoritma SURF dan SIFT. Kemudian visualisasi pencocokan fitur yang berhasil antara dua gambar dan lakukan analisis perbandingan antara dua algoritma ini dalam hal kecepatan, akurasi, dan robustness

1. Metode Penelitian

Scale Invariant Feature Transform (SIFT) Merupakan algoritma yang diperkenalkan oleh David Lowe pada tahun 1999, adalah algoritma yang dirancang untuk mendeteksi dan mendeskripsikan fitur lokal dalam gambar [1]. SIFT memiliki invarian skala, rotasi, dan translasi. SIFT tahan terhadap perubahan iluminasi. Algoritma ini digunakan untuk pengenalan objek berdasarkan pencocokan fitur. Ide utama dari algoritma SIFT adalah untuk menemukan lokal ekstrema dalam skala-ruang gaussian, dan kemudian mengambil titik lokal ekstrema tersebut untuk dijadikan titik fitur yang stabil. Kemudian, karakteristik lokal dari citra diekstraksi di sekitar setiap titik fitur yang stabil dan membentuk deskriptor lokal yang digunakan untuk pencocokan [2]. SIFT terdiri dari empat tahap utama:

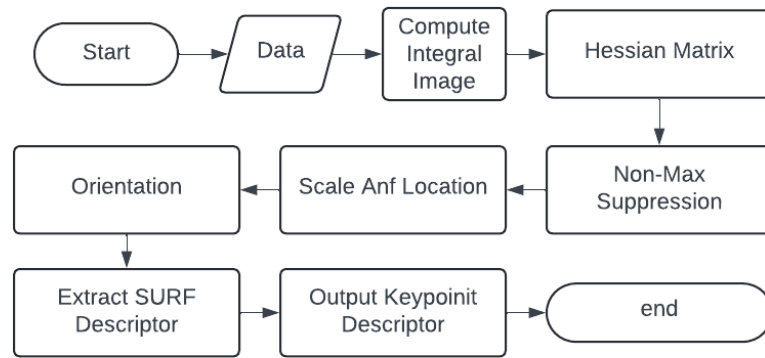
1. Scale-space extrema detection
2. Keypoint localization
3. Orientation assignment
4. Keypoint descriptor



Gambar 1. Flowchart Scale Invariant Feature Transform

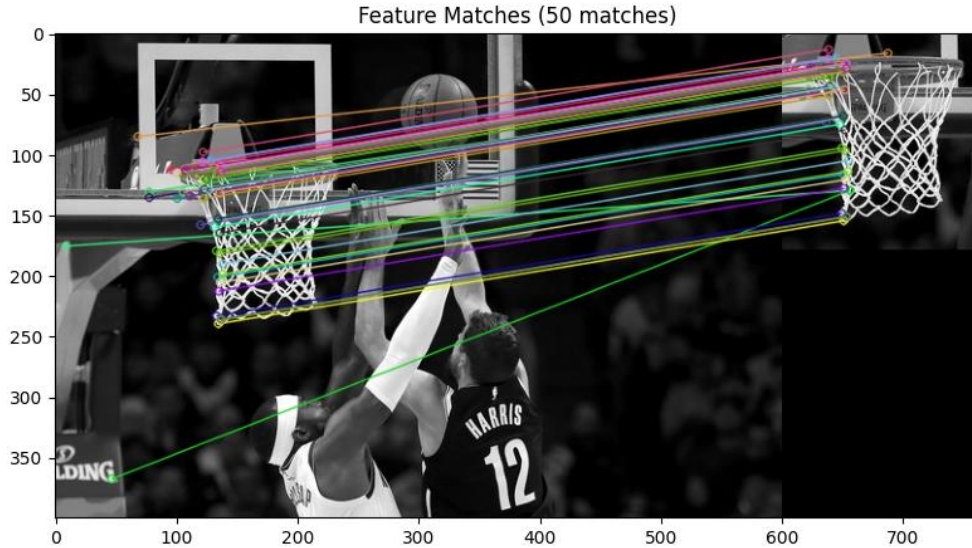
Speeded Up Robust Features (SURF) adalah yang dikembangkan oleh *Bay et al.* pada tahun 2006, adalah algoritma yang terinspirasi oleh SIFT tetapi dirancang untuk kinerja komputasi yang lebih cepat [3]. SURF mendeteksi keypoint citra dan menghasilkan deskriptor. SURF memperoleh keypoints berdasarkan matriks Hessian [4]. Berdasarkan gambar integral dan matriks Hessian, deskriptor keypoints dideteksi menggunakan algoritma SURF. Keypoints dari dua gambar dicocokkan untuk pengenalan. SURF menggunakan beberapa teknik untuk meningkatkan kecepatan:

1. Penggunaan integral image
2. Aproksimasi Laplacian of Gaussian dengan box filter
3. Deskriptor berbasis respons Haar wavelet



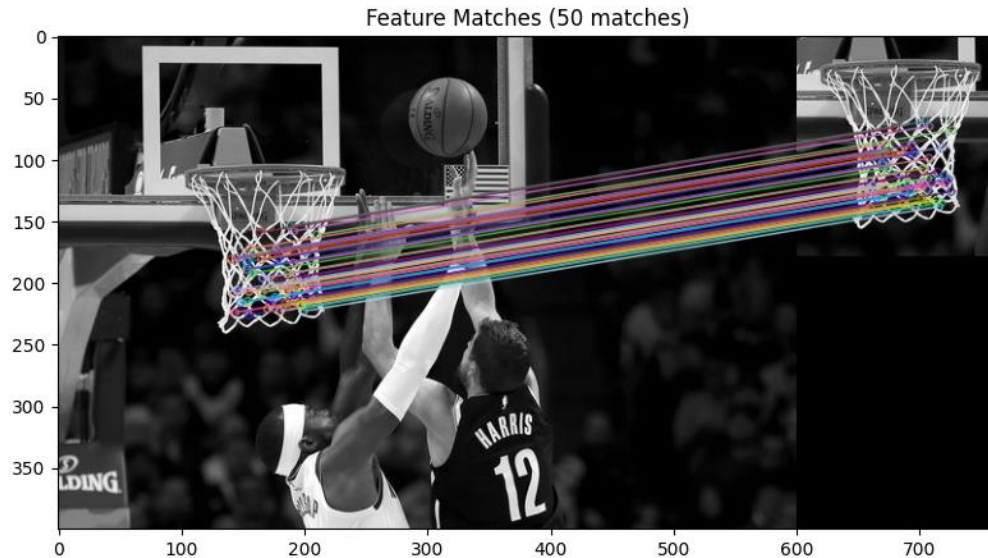
Gambar 2. Flowchart Speeded-Up Robust Features

2. Hasil Dan Pembahasan



Gambar 3. Flowchart Scale Invariant Feature Transform

Pada gambar yang menggunakan metode SIFT **Gambar 3.** terlihat bahwa metode ini menghasilkan pencocokan fitur yang sangat detail dengan menunjukkan banyak garis yang menghubungkan fitur-fitur yang cocok antara dua gambar. SIFT dikenal dengan kemampuannya yang handal dalam mendeteksi fitur yang berbeda-beda pada skala dan rotasi. Fitur-fitur yang terdeteksi umumnya terdistribusi lebih merata di seluruh gambar, terutama di bagian yang memiliki tekstur atau pola yang kuat. Hal ini mencerminkan keandalan metode SIFT dalam mengidentifikasi fitur-fitur yang konsisten meskipun gambar mengalami perubahan sudut pandang, pencahayaan, atau skala.



Gambar 4. Flowchart Speeded-Up Robust Features

Pada gambar yang menggunakan metode SURF **Gambar 4.** meskipun menunjukkan pencocokan fitur yang signifikan, jumlah garis atau pencocokan fitur terlihat lebih sedikit dibandingkan SIFT. Namun, garis-garis yang dihasilkan oleh SURF lebih stabil dan pendek, yang menunjukkan bahwa metode ini lebih cepat dalam mendeteksi fitur utama, meskipun mungkin tidak sedetail SIFT. SURF dirancang untuk menjadi lebih efisien secara komputasi dan biasanya digunakan dalam aplikasi real-time. Namun, ia cenderung kurang sensitif dalam menangkap fitur yang lebih halus dibandingkan SIFT.

perbandingan antara metode **SIFT** dan **SURF** dapat dijelaskan lebih rinci dengan mempertimbangkan waktu pemrosesan, jumlah keypoints yang terdeteksi, dan jumlah pencocokan fitur.

Tabel 1. Hasil Waktu Dari Metode SIFT Dan SURF

Metode: SIFT	Metode: SURF
Waktu pemrosesan : 0.1541 detik	Waktu pemrosesan: 0.0299 detik
Jumlah keypoints (gambar 1): 884	Jumlah keypoints (gambar 1): 500
Jumlah keypoints (gambar 2): 396	Jumlah keypoints (gambar 2): 367
Jumlah pencocokan: 351	Jumlah pencocokan: 225

Berdasarkan hasil analisis perbandingan antara metode SIFT dan SURF pada **Tabel 1**, dapat dilihat perbedaan yang signifikan dari beberapa aspek penting. Dari segi waktu pemrosesan, SIFT membutuhkan waktu 0.1541 detik, sementara SURF hanya memerlukan 0.0299 detik, yang menunjukkan bahwa SURF jauh lebih efisien dalam pemrosesan waktu, sekitar lima kali lebih cepat. Namun, ketika melihat jumlah keypoints yang terdeteksi, SIFT unggul dengan mendeteksi 884 keypoints pada gambar pertama dan 396 keypoints pada gambar kedua, sementara SURF hanya mendeteksi 500 keypoints pada gambar pertama dan 367 keypoints pada gambar kedua. Jumlah pencocokan fitur juga menunjukkan bahwa SIFT lebih akurat dengan menghasilkan 351 pencocokan, dibandingkan dengan **225** pencocokan yang dihasilkan oleh SURF. Dari hasil ini, SIFT terbukti lebih unggul dalam hal ketelitian karena mampu mendeteksi lebih banyak fitur dan mencocokkan lebih banyak titik antara dua gambar, yang sangat penting untuk aplikasi yang memerlukan pengenalan fitur secara detail. Namun, kelemahannya terletak pada waktu pemrosesan yang lebih lambat. Sebaliknya, SURF menunjukkan keunggulan dalam kecepatan, sehingga lebih cocok untuk aplikasi real-time atau yang memerlukan efisiensi waktu. Meskipun demikian, pencocokan fitur yang dihasilkan oleh SURF sedikit lebih rendah dibandingkan SIFT, yang mungkin mengurangi akurasi dalam beberapa kasus yang lebih kompleks. Kesimpulannya, pemilihan metode tergantung pada prioritas antara ketelitian (SIFT) dan efisiensi waktu (SURF).

3. Kesimpulan

Dari hasil dan pembahasan di atas, dapat disimpulkan bahwa **SIFT** lebih unggul dalam hal ketelitian dan cakupan deteksi fitur, dengan kemampuan untuk mendeteksi fitur pada berbagai skala dan rotasi dengan presisi yang lebih tinggi. Metode ini lebih cocok untuk aplikasi yang memerlukan pencocokan fitur yang akurat dan detail. Di sisi lain, **SURF** lebih dioptimalkan untuk kecepatan, dengan hasil yang tetap memadai untuk aplikasi real-time meskipun pencocokan fiturnya tidak sebanyak SIFT. Pemilihan metode tergantung pada kebutuhan spesifik, apakah lebih memprioritaskan ketelitian (SIFT) atau kecepatan (SURF).

Daftar Pustaka

- [1] Hidayat, T., & Teknologi Rekayasa Jaringan Telekomunikasi Jurusan Teknik Elektro Politeknik Negeri Lhokseumawe, P. (2023). PERBANDINGAN ALGORITMA SCALE INVARIANT FEATURE TRANSFORM (SIFT) DAN SPEEDED UP ROBUST FEATURE (SURF) PADA PENGENALAN OBJEK BERBASIS MATLAB. 72 *JURNAL TEKTRO*, 7(1).
- [2] Lowe, D. G. (1999). Object recognition from local scale-invariant features. In Proceedings of the Seventh IEEE International Conference on Computer Vision (Vol. 2, pp. 1150-1157)
- [3] Bay, H., Tuytelaars, T., & Van Gool, L. (2006). SURF: Speeded Up Robust Features. In European Conference on Computer Vision (pp. 404-417). Springer, Berlin, Heidelberg.
- [4] Bay H, Tuytelaars T, Gool LV. Speeded-Up Robust Features (SURF). Computer Vision and Image Understanding. 2008

LAMPIRAN

Link Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def detect_and_match(img1, img2, method='sift'):
    if method == 'sift':
        detector = cv2.SIFT_create()
    elif method == 'surf':
        detector = cv2.ORB_create()
    else:
        raise ValueError("Method should be 'sift' or 'surf'")

    # Deteksi keypoints dan deskriptor
    kp1, des1 = detector.detectAndCompute(img1, None)
    kp2, des2 = detector.detectAndCompute(img2, None)

    # Pencocokan fitur
    if method == 'sift':
        matcher = cv2.BFMatcher()
        matches = matcher.knnMatch(des1, des2, k=2)

        # Aplikasikan rasio tes
        good_matches = []
        for m, n in matches:
            if m.distance < 0.75 * n.distance:
                good_matches.append(m)
    else: # surf
        bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
        good_matches = bf.match(des1, des2)
        good_matches = sorted(good_matches, key=lambda x: x.distance)

    return kp1, kp2, good_matches
```

```

# Fungsi untuk visualisasi
def visualize_matches(img1, kp1, img2, kp2, matches):
    img_matches = cv2.drawMatches(img1, kp1, img2, kp2, matches, None,
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    plt.figure(figsize=(15, 5))
    plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
    plt.title(f'Feature Matches ({len(matches)} matches)')
    plt.axis("on")
    plt.show()

# Data
img1 = cv2.imread('/content/drive/MyDrive/Data
Pengpol/permainanbolabasket.jpeg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('/content/drive/MyDrive/Data Pengpol/ringbasket.jpeg',
cv2.IMREAD_GRAYSCALE)

results = compare_methods(img1, img2)

```

```

# Fungsi untuk analisis perbandingan
def compare_methods(img1, img2):
    methods = ['sift', 'surf']
    results = {}

    for method in methods:
        start_time = cv2.getTickCount()
        kp1, kp2, matches = detect_and_match(img1, img2, method)
        end_time = cv2.getTickCount()

        processing_time = (end_time - start_time) / cv2.getTickFrequency()

        results[method] = {
            'time': processing_time,
            'matches': len(matches),
            'keypoints1': len(kp1),
            'keypoints2': len(kp2)
        }

```

```
        visualize_matches(img1, kp1, img2, kp2, matches[:50])    #
Visualisasi 50 pencocokan terbaik

    return results

# Tampilkan hasil perbandingan
for method, result in results.items():
    print(f"\nMetode: {method.upper()}")
    print(f"Waktu pemrosesan: {result['time']:.4f} detik")
    print(f"Jumlah keypoints (gambar 1): {result['keypoints1']}")
    print(f"Jumlah keypoints (gambar 2): {result['keypoints2']}")
    print(f"Jumlah pencocokan: {result['matches']}")
```